

UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning Project

6CS012

Part – II: Vision Tasks

Student Id: 2408286

Student Name: Arjabi Shrestha

Group: L6CG15

Course: BSc(hons) Computer Science

University Email: A.Shrestha85@wlv.ac.uk

Github link: <https://github.com/Arjabi04/Artificial-Intelligence-and-Machine-Learning/tree/main/Assessment/Visual-task>

Table of Contents

3. Introduction.....	1
4. Dataset.....	2
4.1. Dataset Description	2
4.2. Oversampling to Address Class Imbalance.....	4
4.3. Train / Validation Split.....	4
4.4. Data Augmentation	5
5. Methodology	6
5.1. Baseline CNN.....	6
5.1.1. Model Architecture.....	6
5.1.2. Model Summary Interpretation	7
5.1.3. Model Training	8
5.1.4. Training Curves	8
5.1.5. Model Evaluation	9
5.2. Deeper Architecture with Regularization Layer	11
5.2.1. Model Architecture.....	11
5.2.2. Regularization Techniques	12
5.2.3. Model Summary Interpretation	12
5.2.4. Model Training	13
5.2.5. Training Curves	13
5.2.6. Model Evaluation	14
6. Experiments and Results.....	15
6.1 Baseline vs. Deeper Architecture	15
6.2. Computational Efficiency.....	16
6.3. Training with Different Optimizers.....	17
6.4. Ablation Study	18
6.5 Challenges in Training	19
7. Fine-Tuning or Transfer Learning	20

8. Conclusion and Future Work	23
-------------------------------------	----

Table of Figures

Figure 1: Visualizing one sample per class.....	3
Figure 2: Class distribution before and after oversampling.....	4
Figure 3: Visualizing Augmented Data	5
Figure 4: Training Curves for Baseline CNN	8
Figure 5: Evaluating model.....	9
Figure 6: Training Curves for Deep CNN	13
Figure 7: Confusion Matrix - deep CNN	14
Figure 8: : Curve comparison for Baseline vs. Deeper Architecture.....	15
Figure 9: Training with SGD vs Adam.....	17
Figure 10: Ablation study - no dropout.....	18
Figure 11: Train Accuracy and loss for fine tune model	20
Figure 12: confusion matrix and accuracy for fine tuned model	21
Figure 13: model comparison(Baseline CNN vs deepCNN vs Transfer learning).....	22

1. Title

A Performance Analysis of Deep Learning Architectures for the Classification of 15 Selected Fruit Varieties using CNNs and Transfer Learning

2. Abstract

This project investigates the performance of different deep learning architectures for classifying 15 selected fruit varieties using the Fruits-360 dataset from Kaggle. The main objective was to compare how baseline CNNs, deeper CNN architectures, and transfer learning models perform in image classification tasks. The dataset was preprocessed using resizing, normalization, oversampling, and real-time augmentation techniques such as rotation, zooming, Gaussian blur, and noise injection.

Three different models were trained and evaluated using categorical cross-entropy loss with SGD and Adam optimizers. Model performance was compared using accuracy, validation loss, training time, confusion matrices, and classification reports.

The results showed that deeper CNN architectures significantly improved classification performance compared to the baseline model. Transfer learning using MobileNetV2 achieved the best overall performance, reaching perfect test accuracy while also reducing training time and computational complexity. The project demonstrates that deep learning models are highly effective for fruit image classification tasks and that preprocessing, augmentation, and transfer learning techniques can greatly improve model performance and generalization.

3. Introduction

Image classification involves training machine learning models to automatically identify and classify objects within images. Convolutional Neural Networks (CNNs) have greatly improved image classification performance by allowing models to learn important visual features directly from image data.

Fruit image classification has many real-world applications in agriculture, supermarkets, etc. However, classification can be difficult because many fruits share similar colors, textures, and shapes. Previous studies have shown that deeper CNN architectures and transfer learning models can improve classification accuracy and generalization compared to simple shallow networks.

The main goal of this project was to compare the performance of different deep learning approaches for classifying 15 selected fruit varieties from the Fruits-360 dataset. Three approaches were implemented: a baseline CNN, a deeper CNN with Batch Normalization and Dropout, and a transfer learning model using MobileNetV2. The models were compared using accuracy, validation loss, training time, and computational efficiency to evaluate how architectural complexity and transfer learning affect image classification performance.

4. Dataset

4.1. Dataset Description

Fruits-360 is a publicly available dataset on kaggle containing RGB photographs of fruits, vegetables and nuts. All images are 100×100 pixels and are captured against a uniform white background. Images are already organized into folders according to their class labels, making the dataset suitable for supervised deep learning classification tasks. Due to the large number of available classes and the imbalance between categories, only 15 selected fruit varieties were manually chosen for this project.

Selected Classes:

Number of Classes: 15

Total number of Images: 10,192

Training: 7,635 and Test : 2,557

Classes and their Image count:

SN no	Classes (Kaggle dir name)	Training Data	Test Data
1	Apple (Apple 8)	687	228
2	Banana (Banana Lady Finger 1)	450	152
3	Blueberry (Blueberry 1)	462	154
4	Carambola (Carambola 1)	490	166
5	Dates (Dates 1)	490	166
6	Fig (Fig 1)	702	234
7	Guava (Guava 1)	490	166
8	Kiwi (Kiwi 1)	466	156
9	Mango (Mango 1)	490	166
10	Orange (Orange 1)	479	160

11	Pepper (Pepper 2)	480	158
12	Pineapple (Pineapple 1)	490	166
13	Rambutan (Rambutan 1)	492	164
14	Strawberry (Strawberry 1)	492	164
15	Watermelon (Watermelon 1)	475	157

The dataset has multiple fruit variant folder names such as “Apple 8,” “Kiwi 1,” and “Banana Lady Finger 1.” During extraction, only the base fruit names were kept by removing the variant numbers and extra naming details to create simpler class labels for the classification models.



Figure 1: Visualizing one sample per class

4.2. Oversampling to Address Class Imbalance

One challenge during dataset preparation was class imbalance. To remove class imbalance bias, each training class was oversampled to exactly 1,000 images using mild augmentation. Augmented files are saved to disk with an `aug_` prefix. Oversampling was applied exclusively to the Training directory; the Test directory was never modified.

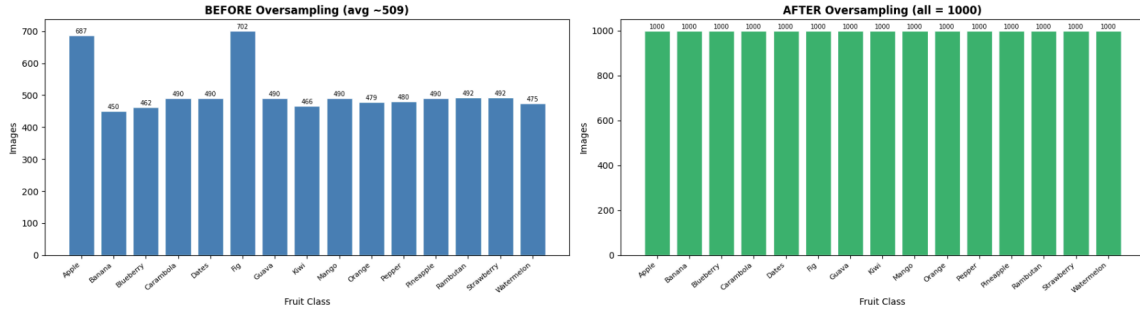


Figure 2: Class distribution before and after oversampling

The images were also normalized by scaling pixel values between 0 and 1 before training. To help the models perform better on unseen data, several augmentation techniques were applied in real time, including rotation, zooming, shifting, horizontal flipping, Gaussian blur, and Gaussian noise. These changes created more variation in the training images and helped reduce overfitting while improving the model's ability to generalize.

4.3. Train / Validation Split

An 80/20 split is applied with ImageDataGenerator. The Fruits-360 dataset already provides a separate Train and Test folder, so the validation split is from the Training directory, ensuring no leakage between validation and test sets. Two separate ImageDataGenerator objects are used: one for training (with full augmentation) and one for validation (rescaling only).

4.4. Data Augmentation

Two levels of augmentation are applied:

1. Mild augmentation techniques were used during oversampling to balance the number of images in each class before training. These augmented images were saved directly into the dataset folders.
2. Real-time augmentation was applied during model training using the training generator. Random transformations were applied to training images during each epoch to help the model generalize better.

The runtime augmentation pipeline included Gaussian blur, Gaussian noise, random rotation, width and height shifting, zooming, horizontal flipping, shearing, and brightness adjustment within the range of 0.85 to 1.15.

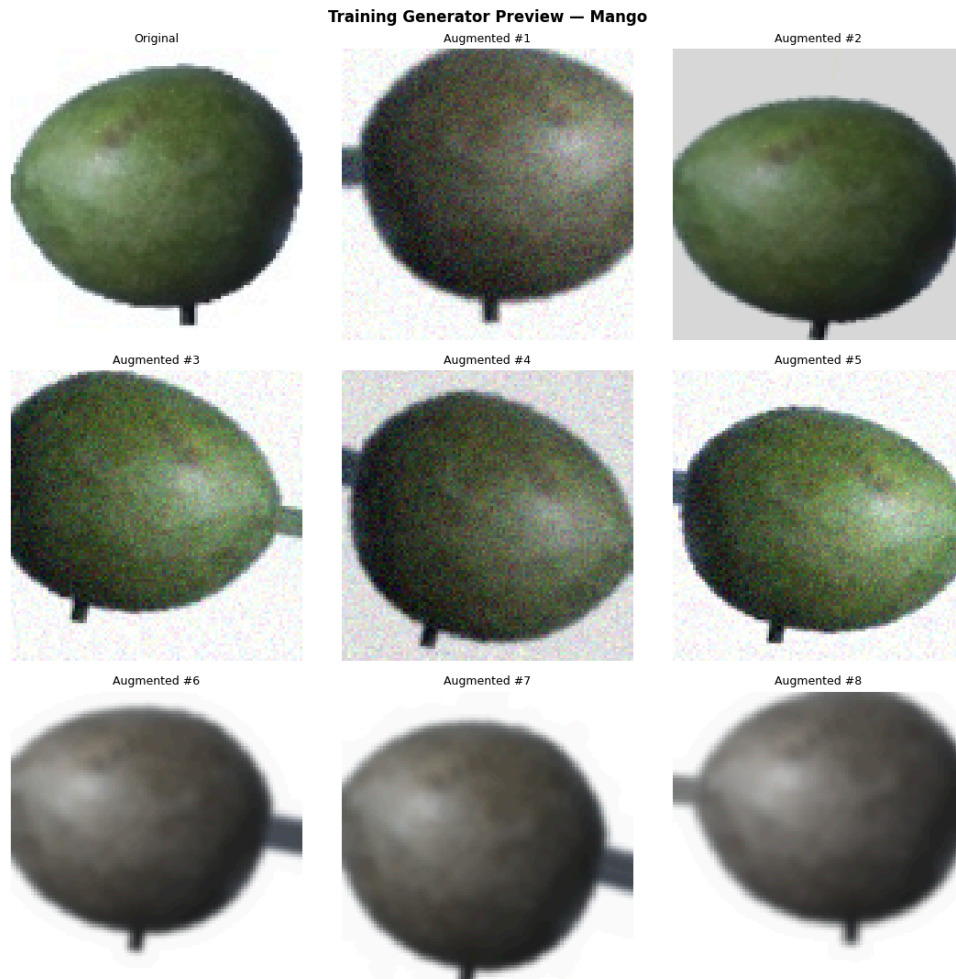


Figure 3: Visualizing Augmented Data

5. Methodology

5.1. Baseline CNN

5.1.1. Model Architecture

Because the fruit images in the dataset were taken on plain white backgrounds and were relatively easy to distinguish, a simple baseline CNN model was developed first to provide a reference point for later comparisons. The model used three convolutional layers with MaxPooling layers placed after each one to help extract important image features while reducing the image dimensions. The output from these layers was then flattened and passed through three dense layers before reaching the final softmax classification layer. Regularization methods were intentionally not included in this baseline model so that the improvements made by deeper and more advanced models could be compared more fairly later in the project.

Component	Details
Input	$100 \times 100 \times 3$
Block 1	Conv2D(8, 3×3 , ReLU, same) -> MaxPool2D(2×2)
Block 2	Conv2D(16, 3×3 , ReLU, same) -> MaxPool2D(2×2)
Block 3	Conv2D(32, 3×3 , ReLU, same) -> MaxPool2D(2×2)
Flatten Layer	Flatten
Dense 1	64 units, ReLU
Dense 2	32 units, ReLU

Dense 3	16 units, ReLU
Output	num_classes units, Softmax
Optimizer	SGD (lr=0.001, momentum=0.2)
Loss	Categorical cross-entropy
Early Stopping	patience=3, monitor=val_loss

All convolutional layers used a 3×3 kernel size to extract image features efficiently while keeping the model lightweight.

ReLU activation was used in the hidden layers to help the model learn complex patterns, while Softmax was used in the output layer for multi-class classification. The number of filters increased from 8 to 16 and then 32 so the model could learn more detailed features in deeper layers.

5.1.2. Model Summary Interpretation

The baseline model contains a relatively small number of trainable parameters, making it lightweight and faster to train. As the image passes through each convolution and pooling block, the spatial size gradually decreases while the feature depth increases, allowing the model to capture more meaningful image patterns. After the final convolutional block, the feature maps are flattened into a one-dimensional vector before being passed into the dense layers for classification.

Since the model uses smaller filter sizes and fewer layers, it works well as a simple starting model but has limited capacity compared to deeper architectures. Overall, the model contains 303,871 trainable parameters.

5.1.3. Model Training

The baseline model was trained for up to 20 epochs with early stopping applied to prevent unnecessary training and reduce overfitting. Early stopping monitored the validation loss with a patience value of 3, and the best model weights were automatically restored after training. All experiments were performed on a Kaggle Notebook using an NVIDIA Tesla T4 GPU, which reduced training time and allowed each epoch to complete in under 60 seconds.

During training, the model gradually improved its ability to classify fruit images. The final training accuracy reached 48.43% with a training loss of 1.3948, while the final validation accuracy reached 52.83% with a validation loss of 1.1445. The validation accuracy is higher than the training accuracy which suggests that the model generalized well to unseen validation data.

5.1.4. Training Curves

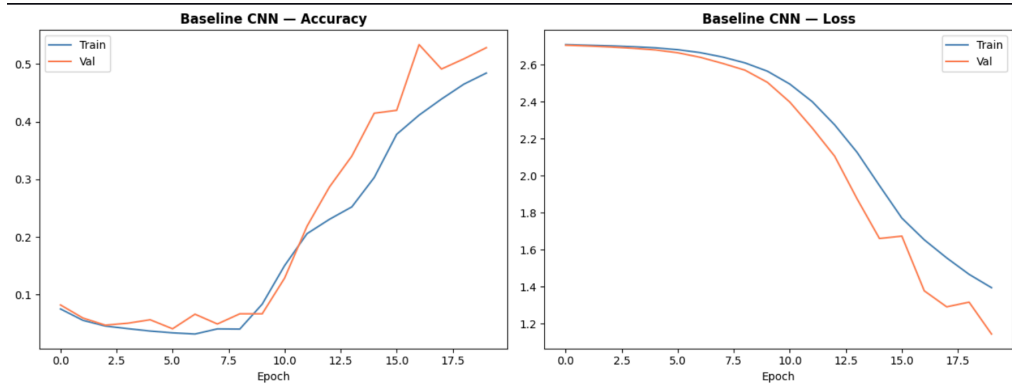


Figure 4: Training Curves for Baseline CNN

The baseline CNN showed steady improvement during training, with both training and validation accuracy increasing across epochs. The model was able to learn meaningful image features from the dataset despite its relatively simple architecture.

The loss curves also decreased consistently throughout training, showing that the model continued improving during the learning process. Validation loss remained lower than

training loss for most epochs, probably because runtime augmentation was applied only to the training data, making the training images more difficult than the validation images. The validation accuracy fluctuated slightly during later epochs, suggesting minor instability as the model approached its learning limit, but no severe overfitting was observed.

5.1.5. Model Evaluation

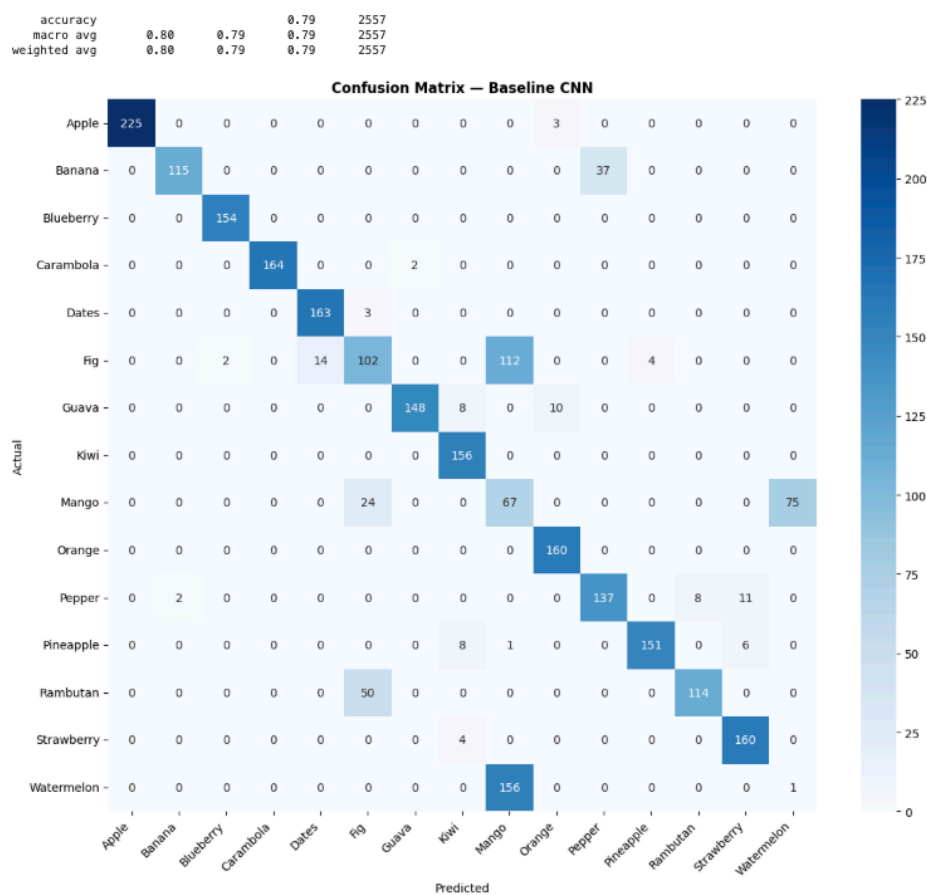


Figure 5: Evaluating model

After training was completed, the baseline CNN model was evaluated on the unseen test dataset to measure its generalization performance. The model achieved a test accuracy of 63.24% with a test loss of 1.0601. The test loss value of 1.0601 indicates that prediction errors were reduced compared to the training phase, showing that the model was able to generalize well to unseen fruit images.

Fruits such as Apple, Blueberry, Carambola, Orange, Pineapple, and Strawberry achieved very high precision and recall scores, indicating strong classification performance on visually distinct classes.

However, the model struggled with more categories such as Fig, Mango, and especially Watermelon, which received very low F1-scores. Moderate confusion was also observed between visually similar shaped fruits such as Banana, Pepper, and Rambutan. Overall, the results show that the baseline CNN was able to learn basic image features effectively, but its relatively shallow architecture limited its performance on more complex and similar-looking classes which could be improved using deeper architectures, regularization, and transfer learning techniques.

5.2. Deeper Architecture with Regularization Layer

5.2.1. Model Architecture

The deeper CNN significantly improved performance compared to the baseline model, achieving higher overall accuracy and more balanced precision and recall across most fruit classes. Difficult categories such as Fig and Watermelon showed major improvement, while confusion between visually similar fruits was reduced. The confusion matrix also contained fewer off-diagonal errors, showing better class separation and feature learning. Overall, the deeper architecture, Batch Normalization, and Dropout helped the model generalize better and correctly classify more challenging fruit images that the baseline CNN previously struggled with.

Component	Details
Input	$100 \times 100 \times 3$
Block 1	Conv2D(8) --> BatchNormalization --> Conv2D(8) --> MaxPool2D(2×2)
Block 2	Conv2D(16) --> BatchNormalization --> Conv2D(16) --> MaxPool2D(2×2) --> Dropout(0.3)
Block 3	Conv2D(32) --> BatchNormalization --> Conv2D(32) --> MaxPool2D(2×2)
Block 4	Conv2D(64) --> BatchNormalization --> Conv2D(64) --> MaxPool2D(2×2) --> Dropout(0.3)
Flatten Layer	Flatten
Dense 1	128 units, ReLU → Dropout(0.5)
Dense 2	64 units, ReLU
Dense 3	32 units, ReLU
Output	NUM_CLASSES units, Softmax
Optimizer	SGD (lr = 0.001)
Loss	Categorical cross-entropy

Early Stopping	patience = 5, monitor = val_loss
-----------------------	----------------------------------

The model was trained using the SGD optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function. Early stopping was also used to stop training when the validation loss stopped improving for 5 epochs, while automatically keeping the best model weights.

5.2.2. Regularization Techniques

Batch Normalization was applied after the first Conv2D in each block. It Normalises activations within each mini-batch, reducing internal covariate shift.

Dropout was applied at two points in the convolutional path (rate = 0.3 after blocks 2 and 4) and once in the dense path (rate = 0.5 after the first dense layer). During training, randomly zeroed activations prevent co-adaptation and force the network to learn redundant representations, improving generalization.

5.2.3. Model Summary Interpretation

The deeper CNN contains more trainable parameters than the baseline model because it uses additional convolutional layers and larger dense layers for feature extraction. These extra layers allow the model to learn more detailed and abstract image patterns from the fruit dataset. Multiple pooling stages gradually reduce the image size before flattening, helping lower computational complexity while still preserving richer feature information through the increased number of channels. Batch Normalization also adds a small number of non-trainable parameters used to stabilize training. Overall, the model contains approximately 379,959 total parameters, including 379,719 trainable parameters and 240 non-trainable parameters.

5.2.4. Model Training

The deeper model is trained for up to 20 epochs with early stopping (patience = 5). The higher patience value accounts for the slower, noisier convergence typical of deeper networks with dropout. Training time increases relative to the baseline due to the larger parameter count and the additional Batch Normalization computations per forward and backward pass. Despite the increased architectural complexity compared to the baseline CNN, GPU acceleration allowed the model to train efficiently, with each epoch taking approximately 49 seconds. The total deep CNN training time was approximately 1010.1 seconds.

The deep CNN showed stronger learning capability than the baseline model. The final training accuracy reached 79.09% with a training loss of 0.5919, while the final validation accuracy reached 90.30% with a validation loss of 0.3013. Compared to the baseline CNN, the deep CNN achieved lower loss values and higher accuracy, indicating that the deeper architecture extracted more meaningful and discriminative image features from the Fruits-360 dataset.

5.2.5. Training Curves

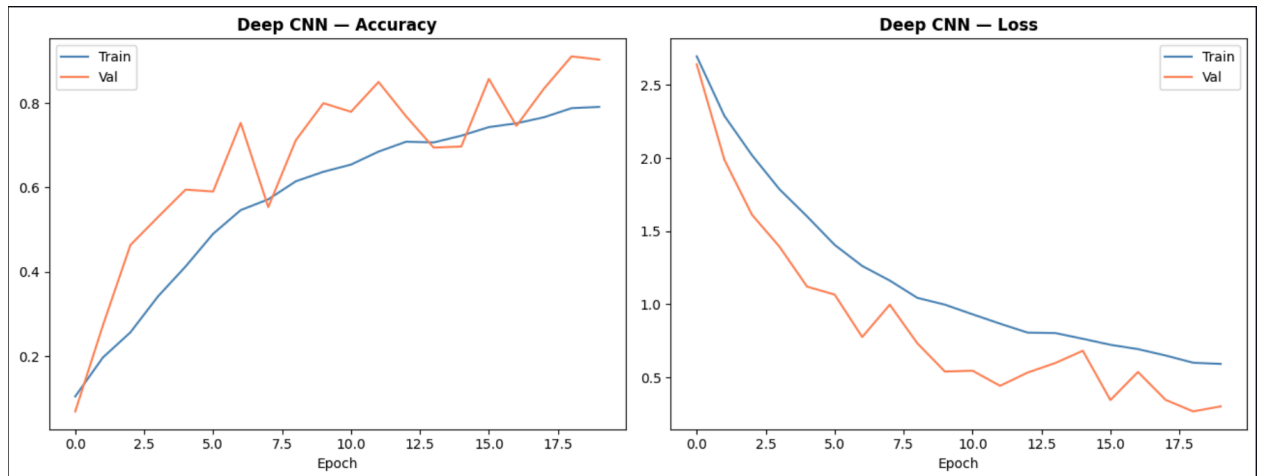


Figure 6: Training Curves for Deep CNN

The deeper CNN showed more stable and improved learning performance compared to the baseline model. Both training and validation accuracy increased steadily across epochs, with validation accuracy reaching around 85–88% near the final epochs. The loss curves were smoother than the baseline model due to Batch Normalization helping stabilize training gradients.

The gap between training and validation accuracy remained small, showing that Dropout and Batch Normalization helped reduce overfitting and improve generalization. Although convergence was slightly slower because of the deeper architecture, the model achieved much higher final accuracy than the baseline CNN.

5.2.6. Model Evaluation

accuracy			0.98	2557
macro avg	0.98	0.98	0.98	2557
weighted avg	0.98	0.98	0.98	2557

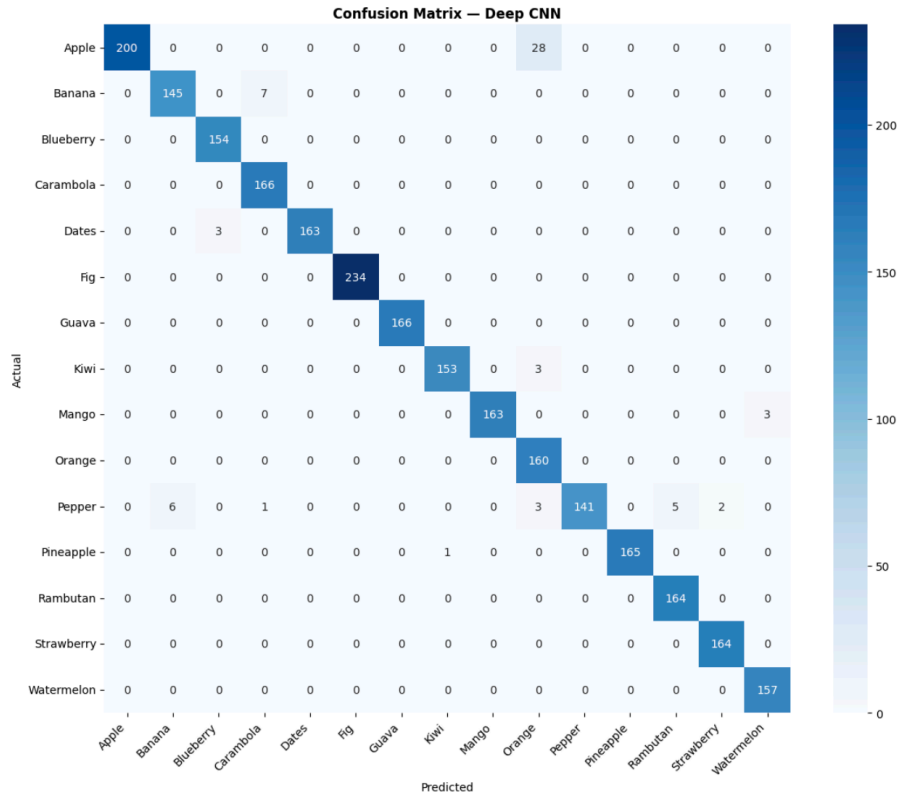


Figure 7: Confusion Matrix - deep CNN

The deeper CNN significantly improved performance compared to the baseline model, achieving higher classification accuracy and stronger overall generalization capability. The model achieved a Deep CNN Test Accuracy of 97.58% with a Test Loss of 0.1745, demonstrating a major improvement over the baseline CNN.

The confusion matrix showed most predictions concentrated along the diagonal, indicating that the majority of images were classified correctly. Compared to the baseline CNN, the deep CNN produced fewer misclassifications and achieved better class separation. Overall, the deeper architecture, Batch Normalization, and Dropout helped the model generalize better and correctly classify more challenging fruit images that the baseline CNN previously struggled with.

6. Experiments and Results

6.1 Baseline vs. Deeper Architecture

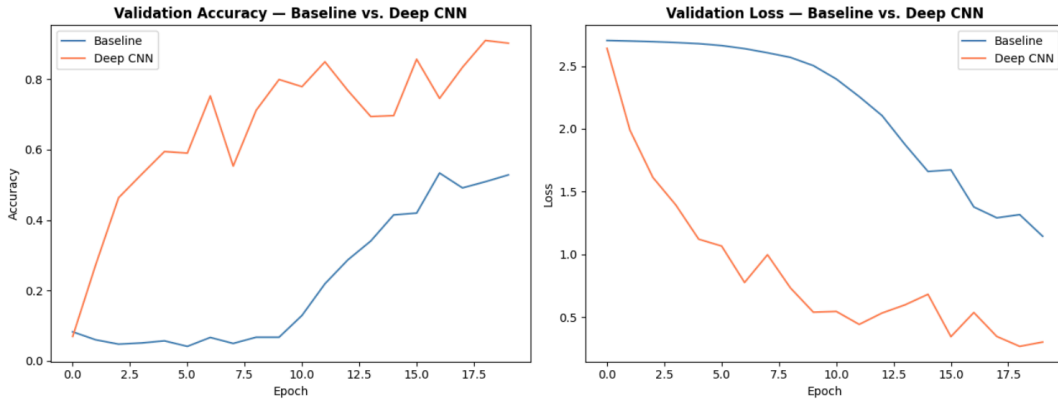


Figure 8: : Curve comparison for Baseline vs. Deeper Architecture

The deeper CNN significantly outperformed the baseline CNN in both validation accuracy and classification performance. The baseline model achieved a best validation accuracy of approximately 63%, while the deeper CNN achieved approximately 88%, showing an improvement of about 24%. The deeper architecture also produced lower validation loss

and more stable learning curves. These improvements were mainly due to the additional convolutional layers, increased filter depth, Batch Normalization, and Dropout regularization, which allowed the model to learn richer and more complex image features.

6.2. Computational Efficiency

```
Baseline training time : 989.5s (16.5 min)
Deep CNN training time : 1010.1s (16.8 min)
Overhead factor        : 1.02x
```

```
Baseline parameters : 303,871
Deep CNN parameters : 379,959
```

The deeper CNN required slightly more computational resources than the baseline model because of its larger parameter count and additional Batch Normalization operations. The baseline model contained approximately 303,871 parameters and completed training in about 16.5 minutes, while the deeper CNN contained approximately 379,959 parameters and trained in about 16.8 minutes.

Despite the increased complexity, the overhead factor was only around 1.02× due to GPU acceleration provided by the NVIDIA Tesla T4 GPU in Kaggle Notebooks.

6.3. Training with Different Optimizers

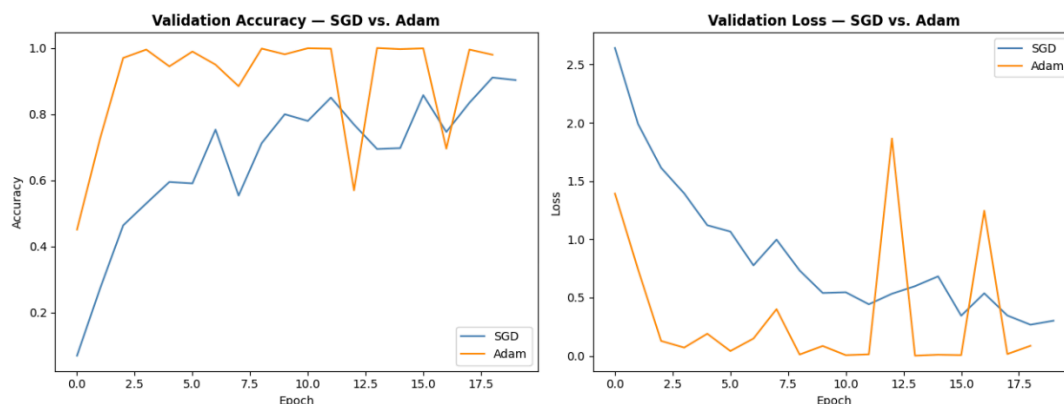
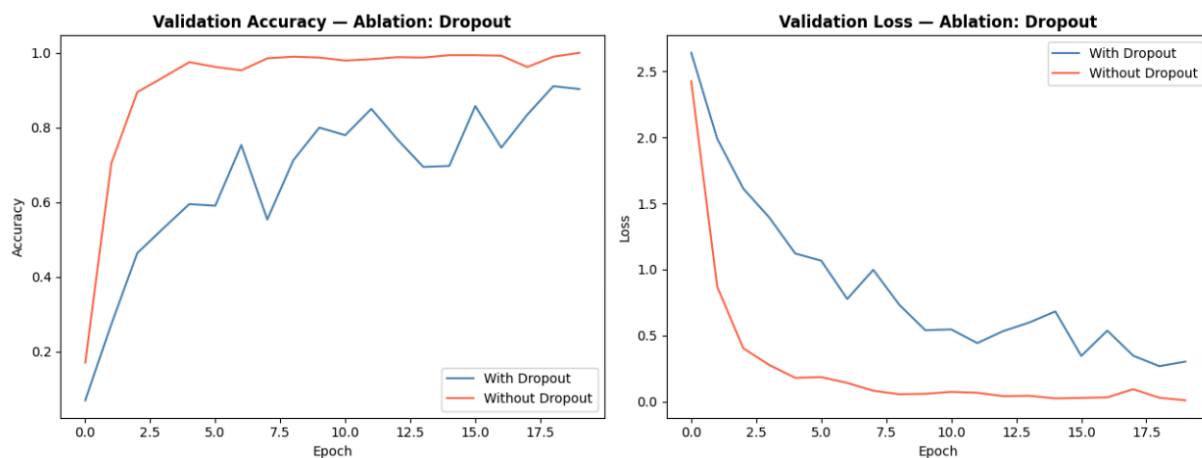


Figure 9: Training with SGD vs Adam

Although Adam achieved perfect test accuracy (100%) with extremely low test loss, this does not necessarily mean the model would generalize perfectly to real-world images. Fruits-360 is a relatively simple and controlled dataset with clean backgrounds and limited variation, making it easier for deep learning models to achieve extremely high performance.

Adam converged much faster than SGD and appeared to fit the dataset very tightly. While this produced excellent test results, it may also indicate a risk of overfitting. In contrast, SGD achieved lower overall accuracy (91.8%) but may generalize more reliably on larger and more diverse datasets because of its slower and more stable optimization behavior. The results suggest that Adam was more effective for maximizing performance on this dataset, while SGD may provide more robust learning behavior for less controlled real-world image classification tasks.

6.4. Ablation Study



Best val accuracy with Dropout : 0.9107

Best val accuracy without Dropout : 1.0000

Figure 10: Ablation study - no dropout

The ablation study was performed by removing Dropout layers from the deeper CNN architecture. Without Dropout, the model achieved extremely high performance, reaching approximately 99.9% validation accuracy and 99.88% test accuracy with very low loss. Training time remained nearly identical at around 1078 seconds.

However, the near-perfect performance without Dropout may indicate overfitting because the model was able to fit the relatively simple Fruits-360 dataset very tightly. With Dropout enabled, the model trained more conservatively and produced slightly lower but probably more generalized performance for real-world image classification tasks.

6.5 Challenges in Training

Several challenges were encountered during training, including dataset imbalance, visually similar fruit classes, and occasional fluctuations in validation accuracy during later epochs. Fruits such as Mango and Watermelon were more difficult to classify because of similarities in color, texture, and shape. Overfitting was also a concern in the deeper architectures, which was addressed using Batch Normalization, Dropout, oversampling, and augmentation techniques.

At the same time, these experiments also helped identify the strengths of the models. The deeper CNN showed strong feature learning ability and was able to classify most fruit categories with very high accuracy, especially on visually distinct classes. GPU acceleration also helped keep training time manageable despite the increased model complexity, allowing experiments to run efficiently within the Kaggle environment.

7. Fine-Tuning or Transfer Learning

Transfer learning was performed using MobileNetV2, a lightweight pre-trained CNN architecture originally trained on large-scale image datasets, making it suitable for this simple Fruits-360 dataset because the images contain clear object-focused features and consistent backgrounds which is recognized by pre-learned models effectively. The original 1,000-class classification head was removed using `include_top=False`, and a new custom classification head was added for the 15 fruit classes in the Fruits-360 dataset. The new head consisted of a `GlobalAveragePooling2D` layer followed by fully connected dense layers and a final softmax output layer.

The transfer learning process was divided into two separate training phases. In Phase 1, the entire MobileNetV2 convolutional base was frozen, and only the newly added dense classification layers were trained. Training remained stable throughout the process, and the best model weights were restored from Epoch 9 using early stopping. In Phase 2, fine-tuning was performed by unfreezing the top 30 layers of MobileNetV2 while keeping the lower layers frozen. This phase further improved performance and produced smoother convergence with extremely low validation loss values.

The final model achieved a training accuracy of 99.07%, a validation accuracy of 99.97%, and a test accuracy of 100% with an extremely low test loss of 0.00063.

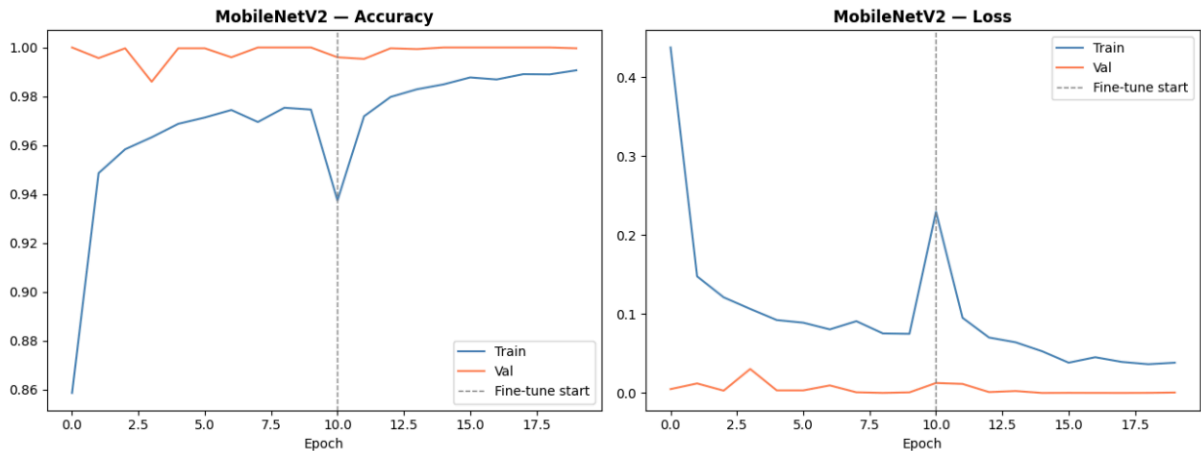


Figure 11: Train Accuracy and loss for fine tune model

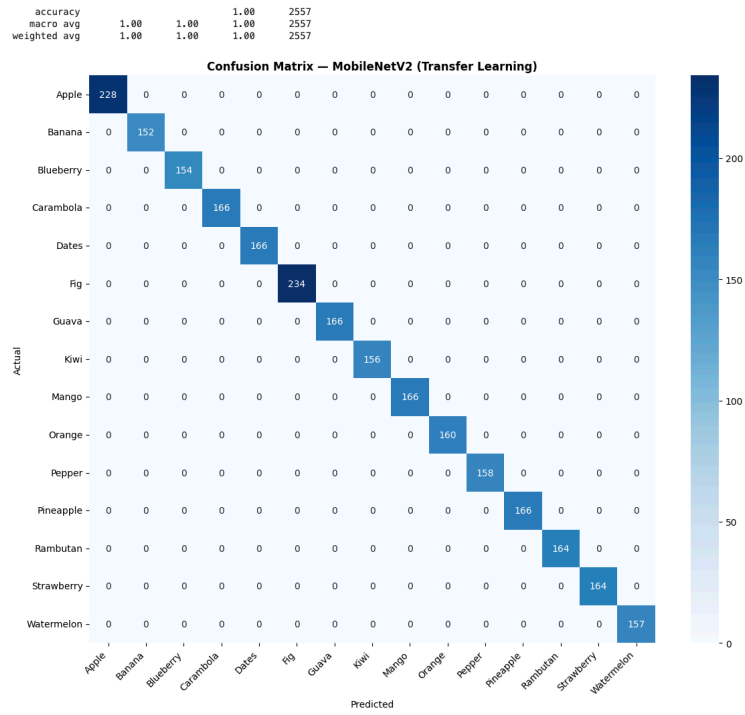


Figure 12: confusion matrix and accuracy for fine tuned model

The fine-tuned MobileNetV2 model achieved the best overall performance among all models, with a test accuracy of 100% and a test loss of 0.00063. These results clearly outperformed both the baseline CNN and the deeper CNN models developed earlier in the project. The training and validation curves showed highly stable learning behavior, with accuracy steadily increasing and loss decreasing throughout both training phases. The confusion matrix also showed no misclassifications, indicating extremely strong feature extraction and classification capability.

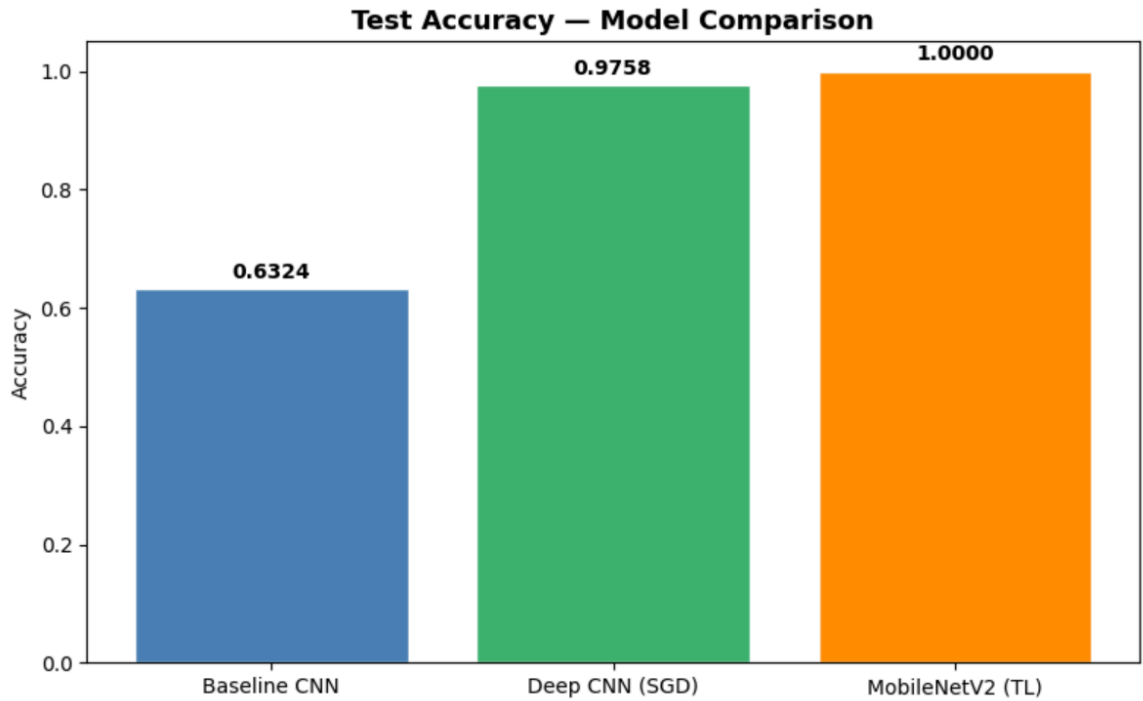


Figure 13: model comparison(Baseline CNN vs deepCNN vs Transfer learning)

Baseline CNN	: 0.6324
Deep CNN (SGD)	: 0.9758
MobileNetV2 (TL)	: 1.0000

Compared to the CNN models trained from scratch, MobileNetV2 converged faster and generalized better while requiring fewer trainable parameters.

Although the model contained over 2.6 million total parameters, most of them remained frozen during training, leaving only around 362,767 trainable parameters which model to achieve very high performance efficiently using previously learned image features from large-scale datasets.

However, the near-perfect performance may also indicate that the Fruits-360 dataset is relatively simple, with clean backgrounds and limited variation. While transfer learning proved extremely effective for this dataset, additional testing on more diverse real-world fruit images would be needed to fully evaluate the model's generalization ability.

8. Conclusion and Future Work

This project compared baseline CNNs, deeper CNN architectures, and transfer learning approaches for classifying 15 selected fruit varieties using the Fruits-360 dataset. The results showed that increasing model depth and using transfer learning significantly improved classification accuracy and feature extraction performance. The baseline CNN provided a simple and computationally efficient starting point, while the deeper CNN improved generalization through additional convolutional layers, Batch Normalization, and Dropout. The best performance was achieved using MobileNetV2 transfer learning, which reached perfect test accuracy while also requiring the shortest training time.

Several trade-offs were observed throughout the experiments. Simpler models trained faster and required fewer computational resources, but they struggled more with visually similar fruit classes. Deeper and transfer learning models achieved substantially higher accuracy but introduced greater architectural complexity and a higher risk of overfitting on simple datasets.

Some limitations also remained in this project. The Fruits-360 dataset contains clean backgrounds and controlled image conditions, which may not fully represent real-world fruit classification scenarios. Near-perfect accuracy may therefore not generalize equally well to images with cluttered backgrounds, lighting variations, or occlusions.

Future work could involve testing the models on more diverse real-world fruit datasets, experimenting with additional transfer learning architectures such as ResNet or EfficientNet, applying further hyperparameter tuning, and evaluating performance under more challenging imaging conditions. Additional techniques such as attention mechanisms or ensemble learning could also be explored to improve robustness and generalization.